

Éco-conception & sobriété numérique

Choix de conception visant à limiter l'empreinte de l'application **Petit Rayon de Soleil** : moins de code, moins de données transférées, moins de ressources serveur mobilisées.

Projet Petit Rayon de Soleil

Auteur de Breyne Hélios

Réf. RNCP CDA — Concevoir l'architecture logicielle

~15 Ko

Poids total du front
(HTML + CSS + JS),
non compressé

0

Image, police web
ou script tiers
chargé côté client

2

Dépendances
runtime seulement
(Express, bcryptjs)

0 W

Consommation
serveur au repos
(hébergement
serverless)

1 Démarche

La sobriété numérique n'a pas été traitée comme une couche ajoutée après coup, mais comme un **principe de conception** présent dès le choix de l'architecture. La règle appliquée : ne mobiliser une ressource (dépendance, requête, octet transféré, calcul serveur) que si elle apporte une valeur réelle à l'utilisateur.

Cette démarche rejoint directement le cahier des charges : l'application a un périmètre volontairement resserré (lire, proposer, mettre en favori des messages bienveillants). Cette simplicité fonctionnelle est aussi ce qui la rend économe.

2 Les six leviers appliqués

DÉPENDANCES

Limitation des dépendances

Le projet ne s'appuie que sur **deux dépendances** d'exécution : `express` (serveur HTTP) et `bcryptjs` (hachage des mots de passe). Les tests utilisent le module natif `node:test` — aucune bibliothèque de test à installer.

Moins de dépendances, c'est un `node_modules` plus petit, moins de code téléchargé et exécuté, une surface d'attaque réduite et une maintenance simplifiée (moins de mises à jour de sécurité en cascade).

Preuve : `package.json` → bloc `dependencies` (Express, bcryptjs, client libSQL pour l'hébergement).

INTERFACE

Interface légère

Le front est écrit en **HTML, CSS et JavaScript natifs**, sans React, Vue ni bundler. Résultat : environ **15 Ko** au total, chargés en trois fichiers seulement, sans étape de compilation.

Aucun framework à télécharger côté navigateur signifie un premier affichage quasi instantané, y compris sur une connexion lente ou un appareil modeste — un gain d'accessibilité autant que de sobriété.

Preuve : dossier `public/` — `index.html` (~4,8 Ko), `style.css` (~3 Ko), `app.js` (~7,9 Ko).

IMAGES

Optimisation des images

L'application ne sert **aucune image** : pas de logo bitmap, pas de bannière, pas d'icône lourde. La mise en forme repose uniquement sur le CSS (couleurs, typographie système, mise en page).

C'est l'optimisation d'image la plus efficace qui soit : celle qu'on n'a pas à charger. Les images sont souvent le premier poste de poids d'une page web ; ici ce poste est nul.

Preuve : aucune balise `<img>` ni `background-image` distante dans `public/` (les fichiers PNG du projet sont les diagrammes du dossier, non servis à l'utilisateur).

HÉBERGEMENT

Hébergement adapté

L'application est déployée sur une infrastructure **serverless** (Vercel), couplée à une base **SQLite / libSQL** (Turso). Le serveur n'est instancié qu'à la réception d'une requête, puis libéré : **aucune ressource n'est consommée au repos**, contrairement à un serveur allumé en permanence.

SQLite est un moteur particulièrement léger (un simple fichier, pas de processus serveur dédié), adapté à la volumétrie réelle du projet. On dimensionne l'infrastructure au juste besoin plutôt que de sur-provisionner.

Preuve : `vercel.json` + `api/index.js` (fonction serverless) ; `db.js` (moteur SQLite / libSQL).

REQUÊTES

Réduction des requêtes inutiles

Les échanges réseau sont réduits au strict nécessaire. Le message d'accueil est obtenu par un unique appel `GET /api/messages/random`, et les requêtes SQL sont **ciblées** : on ne sélectionne que les colonnes utiles et on filtre côté base (`WHERE approved = 1`) plutôt que de tout rapatrier pour trier ensuite.

Chaque requête évitée, c'est de la bande passante et du temps de calcul économisés, côté client comme côté serveur.

Preuve : routes de `server.js` ; requêtes préparées et filtrées de `db.js`.

PÉRIMÈTRE

Application simple et peu consommatrice

Le périmètre fonctionnel est délibérément **minimal et maîtrisé** : consulter des messages, en proposer, gérer ses favoris, modérer côté admin. Pas de fonctionnalité superflue, pas de traitement lourd (pas de média, pas de calcul intensif, pas de tâche de fond permanente).

Une application qui fait peu, mais bien, est structurellement plus sobre qu'une application riche dont la majorité des fonctions restent inutilisées.

Preuve : cahier des charges + liste des routes de l'API (périmètre fermé et documenté).

3 Synthèse

Levier de sobriété	Choix concret dans le projet	Bénéfice
Dépendances	2 dépendances runtime ; tests via <code>node:test</code> natif	Moins de code, moins de failles, maintenance allégée
Interface	HTML/CSS/JS natif, ~15 Ko, sans bundler	Affichage rapide, accessible sur appareil modeste
Images	Aucune image servie, mise en forme CSS seule	Poids de page fortement réduit
Hébergement	Serverless (Vercel) + SQLite/libSQL	Zéro consommation au repos, infra au juste besoin
Requêtes	Appels API minimaux, requêtes SQL ciblées et filtrées	Moins de bande passante et de calcul
Périmètre	Fonctions essentielles uniquement	Consommation structurellement basse

À retenir. La sobriété de *Petit Rayon de Soleil* ne résulte pas d'une contrainte subie mais d'un ensemble de choix d'architecture cohérents : un périmètre simple, un front sans dépendance, une base légère et un hébergement qui ne consomme que sur sollicitation. La conception frugale sert à la fois l'écologie, la performance et l'accessibilité.

Petit Rayon de Soleil — Annexe « Éco-conception et sobriété numérique » du dossier professionnel CDA. Chiffres mesurés sur le code source du projet.